

NBC 节点式 AI 多模态内容创作系统 · 源代码文档说明

用于软件著作权申请 · 提交前后各 30 页源代码

前 30 页：核心架构与引擎层

第 1-3 页 — 类型定义系统

文件：src/types/flow.ts

本文件定义了整个系统的类型基础，包括 11 种节点类型枚举、节点数据接口、节点分组接口以及图的数据结构。

```
import type { Node, Edge } from 'reactflow'

export type NodeType =
  | 'assetInput'
  | 'characterCard'
  | 'sceneCard'
  | 'itemCard'
  | 'prompt'
  | 'script'
  | 'storyboard'
  | 'gptImage2'
  | 'seedance'
  | 'banana'
  | 'output'

export interface NodeGroup {
  id: string
  name: string
  nodeIds: string[]
  color: string
}

export interface NodeData {
  label: string
  _nodeId?: string
  nodeWidth?: number
  nodeHeight?: number
  timelineRowId?: string
  timelineMediaType?: 'image' | 'video'
  _execStatus?: string
  _resultUrl?: string
  _resultUrls?: string[]
  _taskIds?: string[]
  _error?: string
  _localPath?: string
  _ossUrl?: string
  _saveErrors?: string
  _negativePrompt?: string
  _consistencySeed?: number
  batchCount?: number

  // Asset Input
  assetId?: string

  // Character Card
  characterName?: string
  characterAppearance?: string
  characterRefImage?: string
  characterRefImages?: string[]
}
```

```
characterAssetIds?: string[]
characterNames?: string[]
characterAppearances?: string[]
characterCards?: string
characterAssetId?: string
characterFacePrompt?: string
characterBodyPrompt?: string
characterNegativePrompt?: string
characterConsistencySeed?: number
characterTTSVoiceId?: string
characterTTSSpeed?: number
characterTTSPitch?: number

// Scene Card
sceneName?: string
sceneDescription?: string
sceneRefImage?: string
sceneAssetId?: string
sceneAssetIds?: string[]
sceneNames?: string[]
sceneDescriptions?: string[]
sceneRefImages?: string[]

// Item Card
itemName?: string
itemDescription?: string
itemRefImage?: string
itemAssetId?: string
itemAssetIds?: string[]
itemNames?: string[]
itemDescriptions?: string[]
itemRefImages?: string[]

// Prompt
promptText?: string

// Script
scriptText?: string
scriptSceneId?: string
scriptSceneNumber?: number
scriptSceneHeading?: string

// Storyboard
storyboardShotId?: string
storyboardShotNumber?: number
storyboardShotDescription?: string
storyboardShotType?: string
storyboardDialogue?: string
storyboardCharacterIds?: string[]
```

```

storyboardSceneId?: string
storyboardItemIds?: string[]

// GPT Image 2
gptImageAspectRatio?: '1:1' | '16:9' | '9:16' | '4:3' | '3:4'
gptImageNumImages?: number
gptImageQuality?: 'auto' | 'low' | 'medium' | 'high'
gptImageUrls?: string

// Seedance 2.0
seedanceModelId?: string
seedanceMode?: string
seedanceResolution?: '480p' | '720p' | '1080p'
seedanceRatio?: string
seedanceDuration?: number
seedanceGenerateAudio?: boolean
seedanceReturnLastFrame?: boolean
seedanceServiceTier?: 'default' | 'flex'
seedanceWebSearch?: boolean
seedanceReferenceImageCount?: number
seedanceReferenceVideoCount?: number
seedanceReferenceAudioCount?: number

// Banana
bananaModel?: string
bananaAspectRatio?: string
bananaImageSize?: '1K' | '2K' | '4K'
bananaUrls?: string

// Output
outputSaveLocal?: boolean
outputUploadOss?: boolean
outputSyncFeishu?: boolean

// Style
activeStyleId?: string

[key: string]: unknown
}

export type AppNode = Node<NodeData, NodeType>
export type AppEdge = Edge

export interface Workflow {
  id: string
  name: string
  nodes: AppNode[]
  edges: AppEdge[]
  createdAt: string

```

```
updatedAt: string  
}
```

第 4-6 页 — 图论算法引擎

文件: `src/engine/graph.ts`

本文件实现 workflow 执行引擎的图论基础，包括递归上游查找和拓扑排序两种核心算法。

```

import type { AppNode, AppEdge } from '@types/flow'

/**
 * 递归查找指定节点的所有上游节点
 *
 * 从目标节点出发，沿边的反向遍历有向无环图，
 * 收集所有数据来源节点，用于构建执行上下文。
 *
 * @param nodeId 目标节点 ID
 * @param nodes 全图节点列表
 * @param edges 全图边列表
 * @param visited 已访问节点集合（防止环形遍历）
 * @returns 上游节点列表
 */
export function findUpstream(
  nodeId: string,
  nodes: AppNode[],
  edges: AppEdge[],
  visited = new Set<string>()
): AppNode[] {
  if (visited.has(nodeId)) return []
  visited.add(nodeId)
  const upstream: AppNode[] = []
  const incoming = edges.filter(
    (e) => e.target === nodeId && !visited.has(e.source)
  )
  for (const e of incoming) {
    const node = nodes.find((n) => n.id === e.source)
    if (node) {
      upstream.push(node)
      upstream.push(... findUpstream(node.id, nodes, edges, visited))
    }
  }
  return upstream
}

/**
 * 对有向无环图执行拓扑排序
 *
 * 采用 Kahn 算法（BFS 变体）：
 * 1. 统计所有节点的入度
 * 2. 将入度为 0 的节点加入队列
 * 3. 出队 → 输出 → 减少邻居入度 → 入度归零则入队
 * 4. 重复直至队列为空
 *
 * @param nodes 全图节点列表
 * @param edges 全图边列表
 * @returns 拓扑排序后的节点 ID 序列
 */

```

```
export function topoSort(nodes: AppNode[], edges: AppEdge[]): string[] {
  const inDegree = new Map<string, number>()
  const adjacency = new Map<string, string[]>()

  for (const node of nodes) {
    inDegree.set(node.id, 0)
    adjacency.set(node.id, [])
  }

  for (const edge of edges) {
    const targets = adjacency.get(edge.source) || []
    targets.push(edge.target)
    adjacency.set(edge.source, targets)
    inDegree.set(
      edge.target,
      (inDegree.get(edge.target) || 0) + 1
    )
  }

  const queue: string[] = []
  for (const [id, deg] of inDegree) {
    if (deg === 0) queue.push(id)
  }

  const sorted: string[] = []
  while (queue.length) {
    const current = queue.shift()!
    sorted.push(current)
    const neighbors = adjacency.get(current) || []
    for (const neighbor of neighbors) {
      const newDeg = (inDegree.get(neighbor) || 1) - 1
      inDegree.set(neighbor, newDeg)
      if (newDeg === 0) queue.push(neighbor)
    }
  }

  return sorted
}
```

第 7-12 页 — 提示词模板变量解析引擎

文件: `src/engine/promptResolver.ts`

本文件是系统最核心的模块之一，负责提示词模板变量的自动收集、分类、替换。

```

import type { AppNode, AppEdge } from '@types/flow'
import { findUpstream } from './graph'
import { useStyleStore } from '@store/useStyleStore'
import { useAssetStore } from '@store/useAssetStore'

function asStringArray(value: unknown): string[] {
  if (Array.isArray(value))
    return value.filter(
      (item): item is string =>
        typeof item === 'string' && item.trim().length > 0
    )
  if (typeof value === 'string' && value.trim()) return [value]
  return []
}

function isVideoRef(ref: string): boolean {
  const videoExts = [
    '.mp4', '.mov', '.avi', '.webm', '.mkv', '.flv', '.wmv', '.m4v'
  ]
  const lower = ref.toLowerCase()
  return videoExts.some(ext => lower.endsWith(ext))
}

/**
 * 收集上游所有媒体素材的引用路径，并按类型自动分为图片和视频两类。
 *
 * 分类优先级：
 * 1. 通过 useAssetStore 按素材 UUID 查找，读取 asset.type 字段精确判断
 * 2. 如果素材不在 store 中（如直接引用路径），回退到文件后缀名判断
 *
 * @param promptNodeId 提示词节点 ID
 * @param nodes 全图节点
 * @param edges 全图边
 * @returns { imageRefs, videoRefs } 图片和视频的路径列表
 */
export function collectMediaRefsFromUpstream(
  promptNodeId: string,
  nodes: AppNode[],
  edges: AppEdge[]
): { imageRefs: string[]; videoRefs: string[] } {
  const upstream = findUpstream(promptNodeId, nodes, edges)
  const imageRefs: string[] = []
  const videoRefs: string[] = []
  const assets = useAssetStore.getState().assets

  for (const node of upstream) {
    if (node.type === 'assetInput' && node.data.assetId) {
      const assetId = node.data.assetId as string
      const asset = assets.find(

```



```

    a ⇒ a.id ≡ assetId || a.path ≡ assetId
  )
  if (asset) {
    if (asset.type ≡ 'video') {
      videoRefs.push(asset.path || assetId)
    } else {
      imageRefs.push(asset.path || assetId)
    }
  } else if (isVideoRef(assetId)) {
    videoRefs.push(assetId)
  } else {
    imageRefs.push(assetId)
  }
}

if (node.type ≡ 'characterCard') {
  const single = node.data.characterRefImage as string | undefined
  if (single) imageRefs.push(single)
  const multi = node.data.characterRefImages as string[] | undefined
  if (multi) multi.forEach(r ⇒ imageRefs.push(r))
}

if (node.type ≡ 'sceneCard') {
  if (node.data.sceneRefImage)
    imageRefs.push(node.data.sceneRefImage as string)
  asStringArray(node.data.sceneRefImages).forEach(
    (ref) ⇒ imageRefs.push(ref)
  )
}

if (node.type ≡ 'itemCard') {
  if (node.data.itemRefImage)
    imageRefs.push(node.data.itemRefImage as string)
  asStringArray(node.data.itemRefImages).forEach(
    (ref) ⇒ imageRefs.push(ref)
  )
}

}

return { imageRefs, videoRefs }
}

/**
 * 替换提示词中的视频/图片参考变量（不带 {{}} 的纯文本写法）
 */
function resolveMediaVariables(
  text: string,
  imageRefs: string[],
  videoRefs: string[]
): string {
  let result = text

```

```

for (let i = 0; i < Math.max(imageRefs.length, 10); i++) {
  const placeholder = `图片参考${i + 1}`
  if (result.includes(placeholder)) {
    if (i < imageRefs.length) {
      const escaped = placeholder.replace(
        /[.*+?^${}()|[\]\|]/g, '\\$&'
      )
      result = result.replace(
        new RegExp(escaped, 'g'),
        imageRefs[i]
      )
    }
  }
}

for (let i = 0; i < Math.max(videoRefs.length, 10); i++) {
  const placeholder = `视频参考${i + 1}`
  if (result.includes(placeholder)) {
    if (i < videoRefs.length) {
      const escaped = placeholder.replace(
        /[.*+?^${}()|[\]\|]/g, '\\$&'
      )
      result = result.replace(
        new RegExp(escaped, 'g'),
        videoRefs[i]
      )
    }
  }
}

return result
}

/**
 * 替换提示词中的视频/图片参考变量（带 {} 双括号的模板语法写法）
 */
function resolveTemplatesWithMedia(
  text: string,
  promptNode: AppNode,
  nodes: AppNode[],
  edges: AppEdge[]
): string {
  const { imageRefs, videoRefs } = collectMediaRefsFromUpstream(
    promptNode.id, nodes, edges
  )

  let result = text
  for (let i = 0; i < Math.max(imageRefs.length, 10); i++) {
    const key = `{{图片参考${i + 1}}}`

```

```

    if (result.includes(key)) {
      result = result.split(key).join(
        i < imageRefs.length ? imageRefs[i] : ''
      )
    }
  }
}

for (let i = 0; i < Math.max(videoRefs.length, 10); i++) {
  const key = `{{视频参考${i + 1}}}`
  if (result.includes(key)) {
    result = result.split(key).join(
      i < videoRefs.length ? videoRefs[i] : ''
    )
  }
}

return resolveMediaVariables(result, imageRefs, videoRefs)
}

/**
 * 将提示词中的模板变量（{{character}} 等）替换为上游节点的实际值
 */
export function resolveTemplates(
  text: string,
  promptNode: AppNode,
  nodes: AppNode[],
  edges: AppEdge[]
): string {
  const upstream = findUpstream(promptNode.id, nodes, edges)
  const charNode = upstream.find((n) => n.type === 'characterCard')
  const sceneNode = upstream.find((n) => n.type === 'sceneCard')
  const itemNode = upstream.find((n) => n.type === 'itemCard')

  const characterNames = asStringArray(
    charNode?.data.characterNames
  ).length
  ? asStringArray(charNode?.data.characterNames).join('、')
  : ((charNode?.data.characterName as string) || '')

  const characterAppearances = asStringArray(
    charNode?.data.characterAppearances
  ).length
  ? asStringArray(
    charNode?.data.characterAppearances
  ).join('\n')
  : ((charNode?.data.characterAppearance as string) || '')

  const sceneNames = asStringArray(
    sceneNode?.data.sceneNames
  ).length

```

```
? asStringArray(sceneNode?.data.sceneNames).join('、')
: ((sceneNode?.data.sceneName as string) || '')

const sceneDescriptions = asStringArray(
  sceneNode?.data.sceneDescriptions
).length
? asStringArray(
  sceneNode?.data.sceneDescriptions
).join('\n')
: ((sceneNode?.data.sceneDescription as string) || '')

const itemNames = asStringArray(
  itemNode?.data.itemNames
).length
? asStringArray(itemNode?.data.itemNames).join('、')
: ((itemNode?.data.itemName as string) || '')

const itemDescriptions = asStringArray(
  itemNode?.data.itemDescriptions
).length
? asStringArray(
  itemNode?.data.itemDescriptions
).join('\n')
: ((itemNode?.data.itemDescription as string) || '')

let resolved = text
  .replace(/\{\{character\}\}/g, characterNames)
  .replace(
    /\{\{characterAppearance\}\}/g,
    characterAppearances
  )
  .replace(/\{\{scene\}\}/g, sceneNames)
  .replace(
    /\{\{sceneDescription\}\}/g,
    sceneDescriptions
  )
  .replace(/\{\{item\}\}/g, itemNames)
  .replace(/\{\{itemDescription\}\}/g, itemDescriptions)

resolved = resolveTemplatesWithMedia(
  resolved, promptNode, nodes, edges
)

return resolved
}

/**
 * 收集完整的生成提示词
 */
```

```

* 遍历所有上游节点，按类型结构化提取信息，
* 构建完整的 AI 生成提示词上下文。
*/
export function collectPrompt(
  nodeId: string,
  nodes: AppNode[],
  edges: AppEdge[]
): string {
  const upstream = findUpstream(nodeId, nodes, edges)
  const scripts: string[] = []
  const prompts: string[] = []
  const characters: string[] = []
  const scenes: string[] = []
  const items: string[] = []

  for (const node of upstream) {
    if (node.type === 'script' && node.data.scriptText) {
      scripts.push(node.data.scriptText as string)
    }
    if (node.type === 'prompt' && node.data.promptText) {
      prompts.push(
        resolveTemplates(
          node.data.promptText as string,
          node, nodes, edges
        )
      )
    }
    if (node.type === 'characterCard') {
      const parts: string[] = []
      const names = asStringArray(node.data.characterNames)
      const appearances = asStringArray(
        node.data.characterAppearances
      )
      if (names.length || appearances.length) {
        parts.push(`角色名: ${names.join('、')}`)
        if (appearances.length)
          parts.push(
            `外观描述: \n${appearances
              .map((item, index) => `${index + 1}. ${item}`)
              .join('\n')}`
          )
      }
    } else {
      const n = node.data.characterName || ''
      const a = node.data.characterAppearance || ''
      if (n || a) parts.push(`角色名: ${n}\n外观描述: ${a}`)
    }
  }
  const facePrompt = node.data
    .characterFacePrompt as string | undefined
  const bodyPrompt = node.data

```

```

        .characterBodyPrompt as string | undefined
    if (facePrompt || bodyPrompt) {
        parts.push(`\n【生成一致性约束 - 请严格遵循】`)
        if (facePrompt) parts.push(`面部: ${facePrompt}`)
        if (bodyPrompt)
            parts.push(`体型/着装: ${bodyPrompt}`)
        const seed = node.data
            .characterConsistencySeed as number | undefined
        if (seed) parts.push(`推荐随机种子: ${seed}`)
    }
    if (parts.length)
        characters.push(
            `【角色设定】\n${parts.join('\n')}`
        )
}

if (node.type === 'sceneCard') {
    const names = asStringArray(node.data.sceneNames)
    const desc = asStringArray(
        node.data.sceneDescriptions
    )
    if (names.length || desc.length) {
        scenes.push(
            `【场景设定】\n场景名: ${names.join(
                '\n'
            )}\n场景描述: ${desc.join('\n')}`
        )
    }
} else {
    const n = node.data.sceneName || ''
    const d = node.data.sceneDescription || ''
    if (n || d)
        scenes.push(
            `【场景设定】\n场景名: ${n}\n场景描述: ${d}`
        )
}
}

if (node.type === 'itemCard') {
    const names = asStringArray(node.data.itemNames)
    const desc = asStringArray(
        node.data.itemDescriptions
    )
    if (names.length || desc.length) {
        items.push(
            `【物品设定】\n物品名: ${names.join(
                '\n'
            )}\n物品描述: ${desc.join('\n')}`
        )
    }
} else {
    const n = node.data.itemName || ''
    const d = node.data.itemDescription || ''

```

```

        if (n || d)
            items.push(
                `【物品设定】\n物品名: ${n}\n物品描述: ${d}`
            )
        }
    }
}

const parts: string[] = []
if (scripts.length)
    parts.push(
        `【剧本/分镜】\n${scripts.join('\n\n')}`
    )
if (prompts.length)
    parts.push(
        `【提示词】\n${prompts.join('\n\n')}`
    )
if (characters.length)
    parts.push(characters.join('\n\n'))
if (scenes.length) parts.push(scenes.join('\n\n'))
if (items.length) parts.push(items.join('\n\n'))

const activeStyleId = nodes
    .find(n => n.id === nodeId)
    ?.data.activeStyleId as string | undefined
if (activeStyleId) {
    const styles = useStyleStore.getState().styles
    const style = styles.find(s => s.id === activeStyleId)
    if (style?.stylePrompt) {
        parts.push(
            `【风格约束 - 严格遵循】\n${style.stylePrompt}`
        )
        useStyleStore.getState().incrementUsage(activeStyleId)
    }
}

return parts.join('\n\n')
}

export function collectNegativePrompt(
    nodeId: string,
    nodes: AppNode[],
    edges: AppEdge[]
): string {
    const negatives: string[] = []
    const upstream = findUpstream(nodeId, nodes, edges)

    for (const node of upstream) {
        if (node.type === 'characterCard') {

```

```

    const np = node.data
      .characterNegativePrompt as string | undefined
    if (np) negatives.push(np)
  }
}

const activeStyleId = nodes
  .find(n => n.id === nodeId)
  ?.data.activeStyleId as string | undefined
if (activeStyleId) {
  const styles = useStyleStore.getState().styles
  const style = styles.find(s => s.id === activeStyleId)
  if (style?.negativePrompt) {
    negatives.push(style.negativePrompt)
  }
}

return negatives.join(', ')
}

export function collectImageRefs(
  nodeId: string,
  nodes: AppNode[],
  edges: AppEdge[],
  excludeTypes?: string[]
): string[] {
  const refs = new Set<string>()
  const upstream = findUpstream(nodeId, nodes, edges)
  const skip = new Set(excludeTypes || [])

  for (const node of upstream) {
    if (skip.has(node.type || '')) continue
    if (node.type === 'assetInput' && node.data.assetId) {
      refs.add(node.data.assetId as string)
    }
    if (node.type === 'characterCard') {
      const single = node.data
        .characterRefImage as string | undefined
      if (single) refs.add(single)
      const multi = node.data
        .characterRefImages as string[] | undefined
      if (multi) multi.forEach(r => refs.add(r))
    }
    if (node.type === 'sceneCard') {
      if (node.data.sceneRefImage)
        refs.add(node.data.sceneRefImage as string)
      asStringArray(node.data.sceneRefImages).forEach(
        (ref) => refs.add(ref)
      )
    }
  }
}

```



```
}
if (node.type === 'itemCard') {
  if (node.data.itemRefImage)
    refs.add(node.data.itemRefImage as string)
  asStringArray(node.data.itemRefImages).forEach(
    (ref) => refs.add(ref)
  )
}
}

const activeStyleId = nodes
  .find(n => n.id === nodeId)
  ?.data.activeStyleId as string | undefined
if (activeStyleId) {
  const styles = useStyleStore.getState().styles
  const style = styles.find(s => s.id === activeStyleId)
  if (style?.refImages)
    style.refImages.forEach(r => refs.add(r))
}

return Array.from(refs)
}

export function collectConsistencySeed(
  nodeId: string,
  nodes: AppNode[],
  edges: AppEdge[]
): number | undefined {
  const upstream = findUpstream(nodeId, nodes, edges)
  for (const node of upstream) {
    if (node.type === 'characterCard') {
      const seed = node.data
        .characterConsistencySeed as number | undefined
      if (seed) return seed
    }
  }
  return undefined
}
```

第 13-15 页 — 执行上下文构建

文件: `src/engine/executionContext.ts`

```
import type { AppNode } from '@types/flow'
import type { GenerationType } from '@types/generation'
import { useFlowStore } from '@store/useFlowStore'
import { useProjectStore } from '@store/useProjectStore'
import {
  collectPrompt, collectImageRefs,
  collectNegativePrompt, collectConsistencySeed
} from '@engine/promptResolver'
import { generatorRegistry } from '@engine/generatorRegistry'

export interface ExecutionContext {
  nodeId: string
  type: GenerationType
  mediaType: 'image' | 'video'
  label: string
  nodeLabel: string
  prompt: string
  imageRefs: string[]
  negativePrompt: string
  consistencySeed: number | undefined
  batchCount: number
  timelineRowId?: string
  timelineMediaType?: 'image' | 'video'
  projectId: string | null
  genParams: Record<string, unknown>
}

export function createExecutionContext(
  nodeId: string
): ExecutionContext | null {
  const { nodes, edges } = useFlowStore.getState()
  const node = nodes.find(n => n.id === nodeId)
  if (!node) return null

  const type = node.type as GenerationType
  const adapter = generatorRegistry.get(type)
  if (!adapter) return null

  const prompt = collectPrompt(nodeId, nodes, edges)
  const imageRefs = collectImageRefs(
    nodeId, nodes, edges,
    type === 'seedance' ? ['characterCard'] : undefined
  )
  const negativePrompt = collectNegativePrompt(
    nodeId, nodes, edges
  )
  const consistencySeed = collectConsistencySeed(
    nodeId, nodes, edges
  )
}
```

```
const nodeLabel =
  (node.data.label as string) || type
const batchCount =
  (node.data.batchCount as number) || 1

const genParams = adapter.buildParams(node)
genParams['prompt'] = prompt?.slice(0, 200) || ''

return {
  nodeId, type,
  mediaType: adapter.mediaType,
  label: adapter.label,
  nodeLabel,
  prompt, imageRefs,
  negativePrompt, consistencySeed,
  batchCount,
  timelineRowId:
    node.data.timelineRowId as string | undefined,
  timelineMediaType:
    node.data.timelineMediaType as
    'image' | 'video' | undefined,
  projectId:
    useProjectStore.getState().activeProjectId,
  genParams,
}
}
```

第 16-18 页 — workflow 执行引擎

文件: `src/store/useExecutionEngine.ts`

```

import { useFlowStore } from './useFlowStore'
import { useGenerationStore } from './useGenerationStore'
import { useNotificationStore } from './useNotificationStore'
import { useTimelineStore } from './useTimelineStore'
import { topoSort } from '@engine/graph'
import type { GenerationTask } from '@types/generation'
import { generatorRegistry } from '@engine/generatorRegistry'
import { createExecutionContext }
  from '@engine/executionContext'
import {
  onExecutionStart, onTaskFailed,
  onExecutionComplete, onExecutionFinish
} from '@engine/executionLifecycle'
import { commitResults }
  from '@engine/resultPipeline'

let taskCounter = 0
function taskId(): string {
  return `gen_${++taskCounter}_${Date.now()}`
}

/**
 * 执行单个节点的生成任务
 *
 * 1. 构建执行上下文（收集上游数据）
 * 2. 匹配生成器适配器
 * 3. 按批次并发调用 AI API
 * 4. 收集结果 → 提交保存
 */
export async function executeNode(nodeId: string) {
  const ctx = createExecutionContext(nodeId)
  if (!ctx) return

  const adapter = generatorRegistry.get(ctx.type)
  if (!adapter) return

  const { addTask, updateTask } =
    useGenerationStore.getState()
  const allTasks: GenerationTask[] = []

  for (let i = 0; i < ctx.batchCount; i++) {
    const task: GenerationTask = {
      id: taskId(), nodeId,
      type: ctx.type, status: 'running',
      progress: 0,
      startedAt: new Date().toISOString(),
      abortController: new AbortController(),
    }
    addTask(task)
  }

```

```
    allTasks.push(task)
  }

  onExecutionStart(
    ctx, allTasks.map(t => t.id)
  )

  const results = await Promise.allSettled(
    allTasks.map(async (task) => {
      try {
        if (task.abortController?.signal.aborted)
          throw new Error('AbortError')
        const url = await adapter.execute(
          useFlowStore.getState().nodes.find(
            n => n.id === nodeId
          )!,
          ctx.prompt, ctx.imageRefs,
          task.abortController?.signal,
        )
        updateTask(task.id, {
          status: 'completed', progress: 100,
          resultUrl: url,
          completedAt: new Date().toISOString(),
        })
        return url
      } catch (err: any) {
        if (err.message === 'AbortError') {
          updateTask(task.id, {
            status: 'failed',
            error: 'User cancelled',
            completedAt: new Date().toISOString(),
          })
          throw err
        }
        const errorMsg = err.message || String(err)
        updateTask(task.id, {
          status: 'failed', error: errorMsg,
          completedAt: new Date().toISOString(),
        })
        onTaskFailed(ctx, errorMsg)
        throw err
      }
    }),
  )

  const successfulUrls = results
    .filter(
      r => r.status === 'fulfilled' && r.value
    )
```

```

    .map(
      r =>
        (r as PromiseFulfilledResult<string>).value
    )
const failedCount = results.filter(
  r => r.status === 'rejected'
).length

onExecutionComplete(
  ctx, successfulUrls, failedCount,
  allTasks.length
)

if (successfulUrls.length > 0) {
  await commitResults(ctx, successfulUrls)
}

onExecutionFinish()
}

/**
 * 全部运行：拓扑排序后依次执行图中所有生成节点
 */
export async function executeAll() {
  const { nodes } = useFlowStore.getState()
  const genTypes = new Set(generatorRegistry.types)
  const sorted = topoSort(
    nodes, useFlowStore.getState().edges
  )
  for (const nodeId of sorted) {
    const node = nodes.find(n => n.id === nodeId)
    if (node && genTypes.has(node.type || '')) {
      await executeNode(node.id)
    }
  }

  const tracks = useTimelineStore.getState().tracks
  const videoTrack = tracks.find(
    t => t.type === 'video'
  )
  const unfilled = videoTrack
    ? videoTrack.clips.filter(
        c => c.status === 'empty'
      )
    : []
  if (unfilled.length > 0) {
    useNotificationStore
      .getState()
      .addNotification({

```

```
    type: 'warning',
    title: '时间线有未生成片段',
    message:
      `还有 ${unfilled.length} 个片段未生成，请检查时间线`,
  })
}
}

/**
 * 按节点组独立运行：仅执行指定节点列表中的生成节点
 */
export async function executeNodes(
  nodeIds: string[]
) {
  const { nodes, edges } = useFlowStore.getState()
  const idSet = new Set(nodeIds)
  const genTypes = new Set(generatorRegistry.types)
  const sorted = topoSort(nodes, edges).filter(
    id => idSet.has(id)
  )
  for (const nodeId of sorted) {
    const node = nodes.find(n => n.id === nodeId)
    if (node && genTypes.has(node.type || '')) {
      await executeNode(node.id)
    }
  }
}
```

后 30 页：前端界面与交互层

第 19-22 页 — 核心画布状态管理

文件： `src/store/useFlowStore.ts` (节选：核心 CRUD + 剪贴板 + 节点组 + 导出导入逻辑)

```

import { create } from 'zustand'
import {
  type Connection, addEdge,
  applyNodeChanges, applyEdgeChanges,
  type OnNodesChange, type OnEdgesChange,
  type OnConnect,
} from 'reactflow'
import type {
  AppNode, AppEdge, NodeGroup
} from '@types/flow'
import type { NbcFile } from '@types/project'
import { useProjectStore } from './useProjectStore'
import { emitNBCEvent } from '@utils/nbcEvents'
import { useInspirationStore }
  from './useInspirationStore'

const APP_VERSION = '1.0.0'

let autoSaveTimer: ReturnType<typeof setTimeout>
  | null = null
const AUTO_SAVE_DEBOUNCE = 400

function debouncedAutoSave(
  nodes: AppNode[], edges: AppEdge[]
) {
  if (autoSaveTimer) clearTimeout(autoSaveTimer)
  autoSaveTimer = setTimeout(() => {
    const { groups } = useFlowStore.getState()
    useProjectStore
      .getState()
      .saveCurrentData(nodes, edges, groups)
  }, AUTO_SAVE_DEBOUNCE)
}

function normalizeNode(node: AppNode): AppNode {
  const width =
    typeof node.data?.nodeWidth === 'number'
      ? node.data.nodeWidth : undefined
  const height =
    typeof node.data?.nodeHeight === 'number'
      ? node.data.nodeHeight : undefined
  if (!width && !height) return node
  return {
    ... node,
    style: {
      ... (node.style || {}),
      ... (width ? { width } : {}),
      ... (height ? { height } : {}),
    },
  },

```



```

    }
}

function edgeKey(
  edge: Pick<AppEdge, 'source'|'target'|
    'sourceHandle'|'targetHandle'>
): string {
  return `${edge.source}:` +
    `${edge.sourceHandle || ''}→` +
    `${edge.target}:${edge.targetHandle || ''}`
}

function generateNodeId(): string {
  return `node_${Date.now()}_` +
    `${Math.random().toString(36).slice(2, 8)}`
}

const GROUP_COLORS = [
  '#ff6b6b', '#f9ca24', '#4ecdc4', '#a29bfe',
  '#fd79a8', '#6c5ce7', '#00ceec9', '#e67e22'
]

interface FlowStore {
  nodes: AppNode[]
  edges: AppEdge[]
  selectedNodeId: string | null

  viewportCenter: { x: number; y: number }
  setViewportCenter: (
    center: { x: number; y: number }
  ) => void

  clipboardNodes: AppNode[]
  clipboardEdges: AppEdge[]
  copySelectedNodes: (nodeIds: string[]) => void
  pasteNodes: (
    position: { x: number; y: number }
  ) => void

  groups: NodeGroup[]
  addGroup: (name: string, nodeIds: string[]) => void
  removeGroup: (groupId: string) => void
  updateGroupName: (
    groupId: string, name: string
  ) => void
  setGroups: (groups: NodeGroup[]) => void

  onNodesChange: OnNodesChange
  onEdgesChange: OnEdgesChange

```

```

onConnect: OnConnect
addNode: (node: AppNode) => void
addEdges: (edges: AppEdge[]) => void
removeNode: (id: string) => void
removeNodes: (ids: string[]) => void
updateNodeData: (
  id: string, data: Record<string, unknown>
) => void
selectNode: (id: string | null) => void
setNodes: (nodes: AppNode[]) => void
setEdges: (edges: AppEdge[]) => void
loadFromProject: (
  nodes: AppNode[], edges: AppEdge[]
) => void
newBlank: () => void
exportToFile: () => NbcFile
importFromFile: (file: NbcFile) => void
}

export const useFlowStore = create<FlowStore>(
  (set, get) => ({
    nodes: [],
    edges: [],
    selectedNodeId: null,

    viewportCenter: { x: 250, y: 150 },
    setViewportCenter: (center) =>
      set({ viewportCenter: center }),

    clipboardNodes: [],
    clipboardEdges: [],
    copySelectedNodes: (nodeIds) => {
      const { nodes, edges } = get()
      const selectedSet = new Set(nodeIds)
      const copiedNodes = nodes.filter(
        n => selectedSet.has(n.id)
      )
      const copiedEdges = edges.filter(
        e => selectedSet.has(e.source) &&
          selectedSet.has(e.target)
      )
      set({
        clipboardNodes: copiedNodes,
        clipboardEdges: copiedEdges
      })
    },
    pasteNodes: (position) => {
      const {
        clipboardNodes, clipboardEdges,

```

```
    nodes: existingNodes, edges: existingEdges
  } = get()
  if (clipboardNodes.length === 0) return

  const idMap = new Map<string, string>()
  clipboardNodes.forEach(
    n => idMap.set(n.id, generateNodeId())
  )

  const minX = Math.min(
    ...clipboardNodes.map(n => n.position.x)
  )
  const minY = Math.min(
    ...clipboardNodes.map(n => n.position.y)
  )
  const offsetX = position.x - minX
  const offsetY = position.y - minY

  const newNodes: AppNode[] = clipboardNodes.map(
    n => ({
      ...JSON.parse(JSON.stringify(n)),
      id: idMap.get(n.id)!,
      position: {
        x: n.position.x + offsetX,
        y: n.position.y + offsetY,
      },
      data: {
        ...n.data,
        _nodeId: idMap.get(n.id)!,
      },
      selected: false,
    })
  )

  const newEdges: AppEdge[] = clipboardEdges.map(
    e => ({
      ...JSON.parse(JSON.stringify(e)),
      id: `edge_${Date.now()}_${
        `${Math.random().toString(36).slice(2,8)}`,
      },
      source: idMap.get(e.source)!,
      target: idMap.get(e.target)!,
    })
  )

  const allNodes = [ ... existingNodes, ... newNodes ]
  const allEdges = [
    ... existingEdges, ... newEdges
  ]
  set({ nodes: allNodes, edges: allEdges })
```

```

    debouncedAutoSave(allNodes, allEdges)
  },

  groups: [],
  addGroup: (name, nodeIds) => {
    const groups = [ ...get().groups ]
    const colorIndex =
      groups.length % GROUP_COLORS.length
    const group: NodeGroup = {
      id: `group_${Date.now()}_` +
        `${Math.random().toString(36).slice(2,6)}`,
      name, nodeIds,
      color: GROUP_COLORS[colorIndex],
    }
    set({ groups: [ ...groups, group ] })
  },
  removeGroup: (groupId) => {
    set({
      groups: get().groups.filter(
        g => g.id !== groupId
      )
    })
  },
  updateGroupName: (groupId, name) => {
    set({
      groups: get().groups.map(
        g => g.id === groupId
          ? { ...g, name } : g
      )
    })
  },
  setGroups: (groups) => set({ groups }),

  onNodesChange: (changes) => {
    const nodes = (
      applyNodeChanges(
        changes, get().nodes
      ) as AppNode[]
    ).map(normalizeNode)
    set({ nodes })
    debouncedAutoSave(nodes, get().edges)
  },
  onEdgesChange: (changes) => {
    const edges = applyEdgeChanges(
      changes, get().edges
    ) as AppEdge[]
    set({ edges })
    debouncedAutoSave(get().nodes, edges)
  },

```

```

onConnect: (connection: Connection) => {
  const edges = addEdge(
    connection, get().edges
  ) as AppEdge[]
  set({ edges })
  debouncedAutoSave(get().nodes, edges)
},
addNode: (node) => {
  const nodes = [
    ... get().nodes, normalizeNode(node)
  ]
  set({ nodes })
  debouncedAutoSave(nodes, get().edges)
  const pId =
    useProjectStore.getState().activeProjectId
  const nodeLabel =
    (node.data?.label as string) || node.type
  emitNBCEvent(
    'workflow:node:add',
    pId || undefined,
    {
      summary:
        `添加了节点「${nodeLabel}」` +
        `(${node.type})`,
      nodeType: node.type,
      nodeLabel,
    }
  )
},
addEdges: (newEdges) => {
  const existing = get().edges
  const existingKeys = new Set(
    existing.map((edge) => edgeKey(edge))
  )
  const merged = [
    ... existing,
    ... newEdges.filter(
      (edge) =>
        !existingKeys.has(edgeKey(edge))
    ),
  ]
  set({ edges: merged })
  debouncedAutoSave(get().nodes, merged)
},
removeNode: (id) => {
  const oldNode = get().nodes.find(
    (n) => n.id === id
  )
  const nodes = get().nodes.filter(

```

```

    (n) => n.id !== id
  )
  const edges = get().edges.filter(
    (e) => e.source !== id && e.target !== id
  )
  set({ nodes, edges })
  debouncedAutoSave(nodes, edges)
  if (oldNode) {
    const pId =
      useProjectStore.getState().activeProjectId
    const nodeLabel =
      (oldNode.data?.label as string) ||
      oldNode.type
    emitNBCEvent(
      'workflow:node:remove',
      pId || undefined,
      {
        summary:
          `删除了节点「${nodeLabel}」` +
          `(${oldNode.type})`,
        nodeType: oldNode.type,
        nodeLabel,
      }
    )
  }
},
removeNodes: (ids) => {
  const idSet = new Set(ids)
  const nodes = get().nodes.filter(
    (n) => !idSet.has(n.id)
  )
  const edges = get().edges.filter(
    (e) =>
      !idSet.has(e.source) &&
      !idSet.has(e.target)
  )
  const groups = get()
    .groups
    .map(g => ({
      ...g,
      nodeIds: g.nodeIds.filter(
        nid => !idSet.has(nid)
      ),
    })),
    .filter(g => g.nodeIds.length > 0)
  set({ nodes, edges, groups })
  debouncedAutoSave(nodes, edges)
},
updateNodeData: (id, data) => {

```

```
const nodes = get().nodes.map(
  (n) =>
    n.id === id
      ? { ...n, data: { ...n.data, ...data } }
      : n
)
set({ nodes })
debouncedAutoSave(nodes, get().edges)
},
selectNode: (id) =>
  set({ selectedNodeId: id }),
setNodes: (nodes) => {
  const normalized = nodes.map(normalizeNode)
  set({ nodes: normalized })
  debouncedAutoSave(normalized, get().edges)
},
setEdges: (edges) => {
  set({ edges })
  debouncedAutoSave(get().nodes, edges)
},

loadFromProject: (nodes, edges) => {
  const pId =
    useProjectStore.getState().activeProjectId
  let groups: NodeGroup[] = []
  if (pId) {
    const data = useProjectStore
      .getState()
      .loadCurrentData()
    groups = data?.groups || []
  }
  set({
    nodes: nodes.map(normalizeNode),
    edges,
    selectedNodeId: null,
    groups,
  })
},

newBlank: () => {
  set({
    nodes: [], edges: [],
    selectedNodeId: null, groups: [],
  })
},

exportToFile: () => {
  const { nodes, edges, groups } = get()
  const project = useProjectStore
```

```
.getState()
.getActiveProject()
const now = new Date().toISOString()
const pId = project?.id

let timelineData = null
let inspirationData = null
if (pId) {
  try {
    const raw =
      localStorage.getItem(
        `nbc_timeline_v2_${pId}`
      ) ||
      localStorage.getItem(
        `nbc_timeline_${pId}`
      )
    if (raw) timelineData = JSON.parse(raw)
  } catch {}

  try {
    inspirationData = useInspirationStore
      .getState()
      .getProjectData(pId)
  } catch (e) {
    console.error(
      'Failed to get inspiration data:', e
    )
  }
}

emitNBCEvent('workflow:export', pId, {
  summary:
    `导出了工作流` +
    `[ ${project?.name || '未命名'} ]` +
    `( ${nodes.length} 节点, ` +
    `${edges.length} 连线)`,
})
return {
  version: '1.0.0',
  projectName:
    project?.name || '未命名项目',
  nodes, edges,
  metadata: {
    createdAt:
      project?.createdAt || now,
    updatedAt: now,
    nodeCount: nodes.length,
    appVersion: APP_VERSION,
    groups,
```



```
        timeline: timelineData,
        inspiration: inspirationData,
      },
    },
  },
},

importFromFile: (file) => {
  set({
    nodes: (file.nodes || []).map(
      normalizeNode
    ),
    edges: file.edges || [],
    selectedNodeId: null,
    groups: file.metadata?.groups || [],
  })

  const pId =
    useProjectStore.getState().activeProjectId
  if (pId) {
    if (file.metadata?.timeline) {
      try {
        localStorage.setItem(
          `nbc_timeline_v2_${pId}`,
          JSON.stringify(
            file.metadata.timeline
          )
        )
        window.dispatchEvent(
          new Event('timeline-imported')
        )
      } catch {}
    }

    if (file.metadata?.inspiration) {
      try {
        let srcPId = ''
        const itemIdKeys = Object.keys(
          file.metadata.inspiration
            .activeItemIds || {}
        )
        if (itemIdKeys.length > 0) {
          const parts =
            itemIdKeys[0].split('_')
          if (parts.length ≥ 2)
            srcPId = parts
              .slice(0, -1)
              .join('_')
        }
        useInspirationStore
      }
    }
  }
}
```

```
        .getState()
        .importProjectData(
            pId,
            file.metadata.inspiration,
            srcPId || undefined
        )
    } catch (e) {
        console.error(
            'Failed to import inspiration:', e
        )
    }
}
},
})
)
```

第 23-25 页 — 画布编辑器（含节点组视觉叠加层）

文件: `src/components/node-editor/FlowEditor.tsx` (节选: GroupOverlay 组件 + 右键菜单 + 键盘处理)

```

import { useCallback, useRef, useState,
  useEffect, useMemo } from 'react'
import ReactFlow, {
  Background, Controls, MiniMap,
  ReactFlowProvider, useReactFlow,
  useOnSelectionChange, useViewport,
  type Node, type Edge
} from 'reactflow'
import 'reactflow/dist/style.css'
import { useFlowStore } from '@store/useFlowStore'
import { useProjectStore }
  from '@store/useProjectStore'
import { nodeTypes, nodeTypeColors }
  from '@nodes'
import type { NbcFile } from '@types/project'
import type { Asset } from '@types/asset'
import type { AppNode, NodeGroup }
  from '@types/flow'
import {
  Zap, Upload, Download, Trash2,
  FolderOpen, Copy, ClipboardPaste,
  Group, X, Play
} from 'lucide-react'
import {
  executeAll, executeNodes
} from '@store/useExecutionEngine'

let nodeId = 0
function getId(): string {
  return `node_${++nodeId}_${Date.now()}`
}

const GROUP_PADDING = 28
const GROUP_HEADER_HEIGHT = 24

/**
 * 节点组视觉叠层组件
 *
 * 在画布上方渲染每个节点组的虚线边框和交互栏。
 * 包围盒通过组成员节点的位置+尺寸实时计算，
 * 并随 ReactFlow viewport 的缩放和平移同步。
 */
function GroupOverlay() {
  const groups = useFlowStore((s) => s.groups)
  const nodes = useFlowStore((s) => s.nodes)
  const removeGroup = useFlowStore(
    (s) => s.removeGroup
  )
  const viewport = useViewport()

```

```

const groupBounds = useMemo(() => {
  return groups.map(group => {
    const groupNodes = group.nodeIds
      .map(id => nodes.find(n => n.id === id))
      .filter(Boolean) as AppNode[]

    if (groupNodes.length === 0) return null

    let minX = Infinity, minY = Infinity
    let maxX = -Infinity, maxY = -Infinity
    for (const node of groupNodes) {
      const w = (node.data?.nodeWidth as number)
        || (node.style?.width as number) || 220
      const h = (node.data?.nodeHeight as number)
        || (node.style?.height as number) || 140
      minX = Math.min(minX, node.position.x)
      minY = Math.min(minY, node.position.y)
      maxX = Math.max(maxX, node.position.x + w)
      maxY = Math.max(maxY, node.position.y + h)
    }

    return {
      group,
      minX: minX - GROUP_PADDING,
      minY: minY - GROUP_PADDING
        - GROUP_HEADER_HEIGHT,
      maxX: maxX + GROUP_PADDING,
      maxY: maxY + GROUP_PADDING,
    }
  }).filter(Boolean) as {
    group: NodeGroup; minX: number;
    minY: number; maxX: number; maxY: number
  }[]
}, [groups, nodes])

if (groupBounds.length === 0) return null

return (
  <div className="absolute inset-0
    pointer-events-none z-20"
    style={{ overflow: 'visible' }}>
    {groupBounds.map(
      ({ group, minX, minY, maxX, maxY }) => {
        const screenX =
          minX * viewport.zoom + viewport.x
        const screenY =
          minY * viewport.zoom + viewport.y
        const screenW =

```

```

    (maxX - minX) * viewport.zoom
const screenH =
    (maxY - minY) * viewport.zoom
const headerH =
    GROUP_HEADER_HEIGHT * viewport.zoom

return (
  <div key={group.id}
    className="absolute pointer-events-none"
    style={{
      left: screenX, top: screenY,
      width: screenW, height: screenH,
    }}>
    <div
      className="absolute inset-0
        rounded-xl pointer-events-none"
      style={{
        border:
          `2px dashed ${group.color}55`,
        backgroundColor:
          `${group.color}08`,
        boxShadow:
          `0 0 30px ${group.color}10`,
      }}
    />
    <div
      className="absolute top-0 left-0
        right-0 flex items-center gap-1.5
        pointer-events-auto"
      style={{
        height: headerH,
        paddingLeft: 12 * viewport.zoom,
        paddingRight: 6 * viewport.zoom,
        background:
          `linear-gradient(135deg, ` +
          `${group.color}22, ` +
          `${group.color}0D)`,
        borderBottom:
          `1px solid ${group.color}33`,
        borderRadius:
          `12px 12px 0 0`,
        fontSize: 11 * viewport.zoom,
      }}>
      <span
        className="w-2 h-2 rounded-full
          flex-shrink-0"
        style={{
          backgroundColor:
            group.color,

```

```
        width: 8 * viewport.zoom,
        height: 8 * viewport.zoom,
      }}
    />
    <span
      className="font-medium truncate
        flex-1"
      style={{
        color: group.color,
      }}>
      {group.name}
    </span>
    <span
      className="opacity-50
        flex-shrink-0"
      style={{
        color: group.color,
        fontSize: 9 * viewport.zoom,
      }}>
      ({group.nodeIds.length})
    </span>
    <button
      className="flex items-center
        gap-0.5 px-1.5 py-0.5 rounded-md
        flex-shrink-0 transition-colors"
      style={{
        backgroundColor:
          `${group.color}22`,
        color: group.color,
        fontSize: 10 * viewport.zoom,
      }}
      onClick={(e) => {
        e.stopPropagation()
        executeNodes(group.nodeIds)
      }}
      title={
        `运行组「${group.name}」`
      }>
      <Play
        size={10 * viewport.zoom}
      />
      运行
    </button>
    <button
      className="flex-shrink-0
        rounded-full p-0.5
        transition-colors"
      style={{
        color: group.color,
```

```
        opacity: 0.7,
        fontSize:
          10 * viewport.zoom,
      }}
      onClick={e => {
        e.stopPropagation()
        removeGroup(group.id)
      }}
      title="解散组">
      <X
        size={10 * viewport.zoom}
      />
    </button>
  </div>
</div>
)
}
}]
</div>
)
}
```

第 26-28 页 — 提示词节点组件

文件: `src/nodes/PromptNode.tsx` (节选: 素材引用标签 + 上下文刷新 + AI 优化)

```

import { memo, useState, useRef, useMemo }
  from 'react'
import {
  Handle, Position, type NodeProps
} from 'reactflow'
import {
  FileText, Loader2, RefreshCw,
  Sparkles, Check, X, Image, Film, Plus
} from 'lucide-react'
import type { NodeData } from '@types/flow'
import { useFlowStore } from '@store/useFlowStore'
import { useNotificationStore }
  from '@store/useNotificationStore'
import { optimizePrompt }
  from '@api/promptOptimize'
import {
  collectMediaRefsFromUpstream
} from '@engine/promptResolver'
import { AutoTextArea, NodeFrame }
  from './shared'

function getFileName(path: string): string {
  const parts = path
    .replace(/\\/g, '/').split('/')
  const name = parts[parts.length - 1] || path
  return name.length > 28
    ? name.slice(0, 26) + '...' : name
}

function PromptNode({
  id, data, selected
}: NodeProps<NodeData>) {
  const { nodes, edges, updateNodeData } =
    useFlowStore()
  const [optimizing, setOptimizing] =
    useState(false)
  const [optimized, setOptimized] =
    useState<string | null>(null)
  const textareaRef =
    useRef<HTMLTextAreaElement | null>(null)

  const { imageRefs, videoRefs } = useMemo(
    () => collectMediaRefsFromUpstream(
      id, nodes, edges
    ),
    [id, nodes, edges]
  )

  const hasMediaRefs =

```



```
imageRefs.length > 0 || videoRefs.length > 0

const handleRefresh = () => {
  const upstreamNodeIds = new Set<string>()
  edges
    .filter((e) => e.target === id)
    .forEach(
      (e) => upstreamNodeIds.add(e.source)
    )
  const upstreamNodes = nodes.filter(
    (n) => upstreamNodeIds.has(n.id)
  )
  const parts: string[] = []

  for (const un of upstreamNodes) {
    if (un.type === 'characterCard') {
      const names = Array.isArray(
        un.data.characterNames
      )
        ? un.data.characterNames.join(', ')
        : ((un.data.characterName as string)
            || '')
      const desc = Array.isArray(
        un.data.characterAppearances
      )
        ? un.data.characterAppearances
            .join('\n')
        : ((un.data.characterAppearance
            as string) || '')
      if (names || desc)
        parts.push(
          `角色「${names}」: ${desc}`
        )
    } else if (un.type === 'sceneCard') {
      const names = Array.isArray(
        un.data.sceneNames
      )
        ? un.data.sceneNames.join(', ')
        : ((un.data.sceneName as string)
            || '')
      const desc = Array.isArray(
        un.data.sceneDescriptions
      )
        ? un.data.sceneDescriptions
            .join('\n')
        : ((un.data.sceneDescription
            as string) || '')
      if (names || desc)
        parts.push(
```

```

        `场景「${names}」: ${desc}`
    )
} else if (un.type === 'itemCard') {
    const names = Array.isArray(
        un.data.itemNames
    )
        ? un.data.itemNames.join(', ')
        : ((un.data.itemName as string)
            || '')
    const desc = Array.isArray(
        un.data.itemDescriptions
    )
        ? un.data.itemDescriptions
            .join('\n')
        : ((un.data.itemDescription
            as string) || '')
    if (names || desc)
        parts.push(
            `物品「${names}」: ${desc}`
        )
} else if (un.type === 'script') {
    const text =
        (un.data.scriptText as string) || ''
    if (text)
        parts.push(`[剧本]\n${text}`)
} else if (un.type === 'storyboard') {
    const desc =
        (un.data.storyboardShotDescription
            as string) || ''
    const shotType =
        (un.data.storyboardShotType
            as string) || ''
    const dialogue =
        (un.data.storyboardDialogue
            as string) || ''
    parts.push(
        `[分镜 ${shotType}]\n${desc}` +
        `${dialogue}
        ? `\\n对白: 「${dialogue}」` : ''`
    )
}
}
}

if (!parts.length && !hasMediaRefs) {
    useNotificationStore
        .getState()
        .addNotification({
            type: 'warning',
            title: '无上游信息',

```

```

        message:
          '请先连入三卡、场次、分镜或素材节点。',
        })
      return
    }

    if (hasMediaRefs) {
      if (imageRefs.length > 0) {
        const names = imageRefs.map(
          (ref, i) =>
            `{{图片参考${i + 1}}}` +
            `${getFileName(ref)}`
        ).join('\n')
        parts.push(
          `【可用图片素材】\n${names}`
        )
      }
      if (videoRefs.length > 0) {
        const names = videoRefs.map(
          (ref, i) =>
            `{{视频参考${i + 1}}}` +
            `${getFileName(ref)}`
        ).join('\n')
        parts.push(
          `【可用视频素材】\n${names}`
        )
      }
    }

    updateNodeData(
      id, { promptText: parts.join('\n\n') }
    )
  }

  /**
   * 在光标位置插入变量名
   */
  const insertRef = (variableName: string) => {
    const currentText =
      (data.promptText as string) || ''
    const ta = textareaRef.current
    if (ta) {
      const start =
        ta.selectionStart ?? currentText.length
      const end =
        ta.selectionEnd ?? currentText.length
      const newText =
        currentText.slice(0, start) +
        variableName +

```

```
        currentText.slice(end)
      updateNodeData(
        id, { promptText: newText }
      )
      requestAnimationFrame(() => {
        ta.focus()
        const newPos =
          start + variableName.length
        ta.setSelectionRange(newPos, newPos)
      })
    } else {
      updateNodeData(id, {
        promptText: currentText + variableName
      })
    }
  }
}

// ... handleOptimize (AI 优化)
// ... JSX 渲染: 标签按钮 + AutoTextArea
// ... 优化结果预览卡片
}

export default memo(PromptNode)
```

第 29-30 页 — Electron 主进程与安全桥接

文件: `electron/main.ts` (节选: `nbc://` 协议注册 + `fs:scanDirectory` IPC)

```
import {
  app, BrowserWindow, dialog,
  ipcMain, protocol, net, shell
} from 'electron'
import * as path from 'path'
import * as fs from 'fs'

let mainWindow: BrowserWindow | null = null

function createWindow() {
  mainWindow = new BrowserWindow({
    width: 1600, height: 1000,
    minWidth: 1024, minHeight: 600,
    webPreferences: {
      preload: path.join(
        __dirname, 'preload.js'
      ),
      contextIsolation: true,
      nodeIntegration: false,
    },
  })
  mainWindow.loadURL('nbc://app/index.html')
}

app.whenReady().then(() => {
  protocol.handle('nbc', (request) => {
    const url = request.url
    // ...
    const u = new URL(url)
    if (u.hostname === 'assets') {
      const targetPath =
        u.searchParams.get('path')
      if (targetPath &&
        fs.existsSync(targetPath)) {
        return serveFile(targetPath)
      }
    }
    // ... Serving dist files
  })
  createWindow()
})

// IPC: scanDirectory
ipcMain.handle(
  'fs:scanDirectory',
  async (_, dirPath: string) => {
    try {
      return fs.readdirSync(dirPath)
        .filter(f =>
```

```

    ['.png', '.jpg', '.jpeg', '.gif',
     '.webp', '.mp4', '.webm', '.mov',
     '.avi']
    .includes(
      path.extname(f).toLowerCase()
    )
  )
  .map(f => {
    const fp = path.join(dirPath, f)
    const st = fs.statSync(fp)
    const ext =
      path.extname(f).toLowerCase()
    const isV =
      ['.mp4', '.webm', '.mov', '.avi']
        .includes(ext)
    const isImage =
      ['.png', '.jpg', '.jpeg',
       '.gif', '.webp'].includes(ext)
    const fileUrl =
      `nbc://assets/` +
      `${encodeURIComponent(f)}` +
      `?path=` +
      `${encodeURIComponent(fp)}`
    return {
      id: fp, name: f,
      type: isV
        ? 'video' : 'image',
      path: fileUrl,
      thumbnailPath: isImage
        ? fileUrl : undefined,
      size: st.size,
      createdAt:
        st.birthtime.toISOString(),
      tags: ['本地'],
    }
  })
} catch { return [] }
}
)

// IPC: openDirectory
ipcMain.handle(
  'dialog:openDirectory', async () => {
    if (!mainWindow) return null
    const result =
      await dialog.showOpenDialog(
        mainWindow,
        { properties: ['openDirectory'] }
      )
  }
)

```

```
    return result.canceled
      ? null : result.filePaths[0]
  }
}
```

文件: `electron/preload.ts` (安全桥接层)

```
import { contextBridge, ipcRenderer }
  from 'electron'

contextBridge.exposeInMainWorld(
  'electronAPI',
  {
    openDirectory: () =>
      ipcRenderer.invoke(
        'dialog:openDirectory'
      ),
    scanDirectory: (dirPath: string) =>
      ipcRenderer.invoke(
        'fs:scanDirectory', dirPath
      ),
    readFile: (filePath: string) =>
      ipcRenderer.invoke(
        'fs:readFile', filePath
      ),
    apiFetch: (req: any) =>
      ipcRenderer.invoke(
        'api:fetch', req
      ),
    saveFile: (
      filename: string,
      data: string, dir?: string
    ) =>
      ipcRenderer.invoke(
        'save:local',
        filename, data, dir
      ),
    loadFromFile: () =>
      ipcRenderer.invoke(
        'project:loadFromFile'
      ),
    saveToFile: (
      json: string, name: string
    ) =>
      ipcRenderer.invoke(
        'project:saveToFile',
        json, name
      ),
    // ... 30+ additional IPC methods
  }
)
```


源代码文档验证信息

项目	内容
前 30 页覆盖文件	types/flow.ts , engine/graph.ts , engine/promptResolver.ts , engine/executionContext.ts , store/useExecutionEngine.ts
后 30 页覆盖文件	store/useFlowStore.ts , components/node-editor/FlowEditor.tsx , nodes/PromptNode.tsx , electron/main.ts , electron/preload.ts
总行数 (前后 60 页)	约 2000+ 行核心代码
代码格式	TypeScript 5, 缩进 2 空格, 单引号, 分号
编码	UTF-8

文档版本：V1.0.5 · 最后更新：2026年5月